

14강. 마이크로서비스 구현

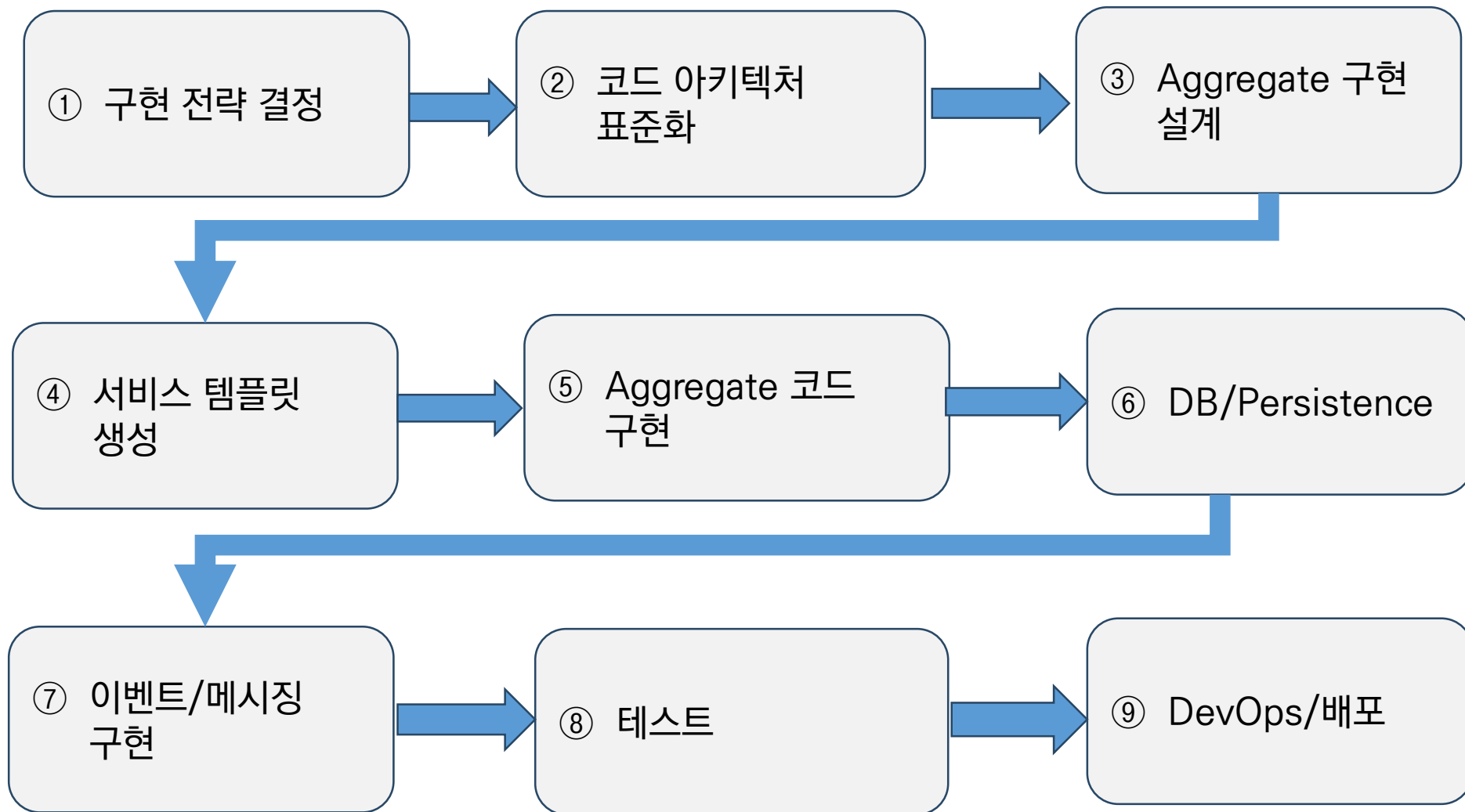


한국공학대학교
TECH UNIVERSITY OF KOREA

이영곤



구현 프로세스



개념

- ✓ DDD 설계 결과를 실제 시스템으로 만들기 전에, 전체 개발팀이 어떤 방식으로 구현할지 정하는 최상위 기술 방향 결정

해야 할 일

- ✓ 아키텍처 스타일 결정
 - Layered
 - Hexagonal
 - Clean
 - Hybrid

아키텍처 스타일

- ✓ 서비스를 어떤 계층과 의존성 구조로 구현할 것인가?

대표 스타일

- ✓ Layered Architecture (계층형)

Controller



Service



Repository



DB

개념	기능을 계층별 책임으로 분리
특징	- 단순 - 이해 쉬움 - 빠른 개발 - Spring Boot 기본 구조와 유사
장점	- 학습 쉬움 - 생산성 높음 - CRUD 적합 - 구현 속도 빠름
단점	- Service 비대화 가능 - 도메인 규칙 분산 - 외부 기술 의존성 증가 - 복잡한 ERP엔 구조 약화 가능

대표 스타일

✓ Hexagonal Architecture (Ports & Adapters)

Input Adapter



Application



Domain



Output Port



Adapter

개념	• 도메인을 중심에 두고 외부 기술을 포트/어댑터로 분리
특징	• 도메인 중심 • 외부 기술 분리 • Port/Adapter 구조 • 테스트 용이
장점	• 도메인 보호 • DB/Kafka/API 교체 용이 • 유지보수 강함 • 복잡한 비즈니스 적합
단점	• 코드량 증가 • 복잡성 증가 • 교육 비용 높음 • 과설계 위험

대표 스타일

✓ Hexagonal Architecture (Ports & Adapters)

- Port: 애플리케이션이 외부와 대화하기 위한 인터페이스
- Adapter: Port를 실제 기술로 구현한 것

```
public interface OrderRepositoryPort {  
    void save(Order order);  
    Optional<Order> findById(Long id);  
}
```

```
@Repository  
public class OrderJpaAdapter implements OrderRepositoryPort {  
  
    private final SpringDataOrderRepository repository;  
  
    @Override  
    public void save(Order order) {  
        repository.save(OrderEntity.from(order));  
    }  
}
```

대표 스타일

✓ Clean Architecture

Framework



Adapters



Use Cases



Entities

개념	모든 의존성은 내부(Use Case/Entity)로 향해야 한다.
특징	- Uncle Bob 구조: 비즈니스 규칙(도메인)을 프레임워크, DB, UI, 외부 시스템으로부터 분리 - 매우 엄격한 의존성 역전 - Use Case 중심
장점	- 높은 순수성 - 강한 테스트성 - 구조 명확
단점	- 실무에서 다소 무거움 - 구현 복잡 - 코드량 많음

대표 스타일

✓ Hybrid Architecture

기본 Layered
+ 핵심 영역 Hexagonal
+ Event Driven

개념	• 복잡도에 따라 구조를 혼합 적용
특징	• 상황 및 환경에 따라 아키텍처 유연 적용
장점	• 실용주의 • 과설계 방지 • 핵심 영역 보호 • 현실성
단점	• 복잡한 구조

아키텍처 스타일 비교

스타일	복잡성	생산성	유지보수	도메인 보호	추천 상황
Layered	낮음	높음	중간	낮음	단순 CRUD
Hexagonal	높음	중간	높음	높음	핵심 비즈니스
Clean	매우 높음	낮음	매우 높음	매우 높음	고복잡도
Hybrid	중간	높음	높음	높음	현실적 기업

도메인 모델과 JPA Entity 관계 결정

방식	설명
Domain = JPA Entity	도메인 클래스에 @Entity 사용 • 개발 용이, JPA/Spring과 잘맞음 • 도메인 모델이 ORM 기술에 종속 • 도메인 로직이 영속성 컨텍스트에 의존
Domain/JPA 분리	순수 Domain과 Persistence Entity 분리 • 순수 도메인 유지 가능 • 테스트 쉬움 • 기술교체 용이 • 개발 난이도 상승
절충형	핵심 도메인은 분리, 단순 CRUD는 통합 • 핵심 비즈니스 보호 가능 • 기준이 애매해질 수 있음 • 구조 일관성 깨질 수 있음

서비스간 통신 전략 결정

- ✓ 동기 API
- ✓ 비동기 이벤트
- ✓ 혼합 방식

- 즉시 응답 필요 → REST API
- 변경 사실 전파 → Kafka Event
- 여러 서비스 상태 전이 → Saga/Event
- 조회 성능 최적화 → CDC

이벤트 전략 결정

- ✓ Domain Event와 Integration Event 구분
 - Domain Event: Aggregate 내부 이벤트
 - Integration Event: 외부 서비스와 통합하기 위해 발행하는 이벤트
- ✓ Topic 명명 규칙
 - 도메인.행위.버전. 예: product.product-registered.v1
 - 파티션 분할 기준, 순서 보장, 보존 기간 등 결정
- ✓ 이벤트 버전 관리
 - 이벤트 스키마내에서 Version 관리
- ✓ Outbox 적용 여부
- ✓ Retry/DLQ 원칙

트랜잭션 전략 결정

- ✓ 서비스 내부 트랜잭션
 - 서비스 내부는 ACID, 서비스 간은 Eventual Consistency
- ✓ Aggregate 단위 저장
 - Aggregate Root 기준 저장
- ✓ 분산 트랜잭션 금지 여부
 - 서비스 간 글로벌 트랜잭션을 허용할 것인가?
- ✓ Saga 사용 기준
 - Choreography ? Orchestrator ?
- ✓ 보상 트랜잭션 기준: Rollback 보장? 반영 보장?
- ✓ Outbox 사용 기준

개념

- ✓ 구현 전략에서 정한 아키텍처 스타일을 개발자가 실제 코드에 동일하게 적용하도록 만드는 개발 표준 정의 단계
- ✓ 아래 사항들을 결정:
 - 패키지 구조
 - 계층 구조
 - 클래스 역할
 - DTO/Command 규칙
 - Repository 규칙
 - Event 발행 방식
 - 예외 처리
 - 테스트 방식

패키지 구조

- ✓ 각 서비스 내부를 어떻게 구현할 것인가 정의

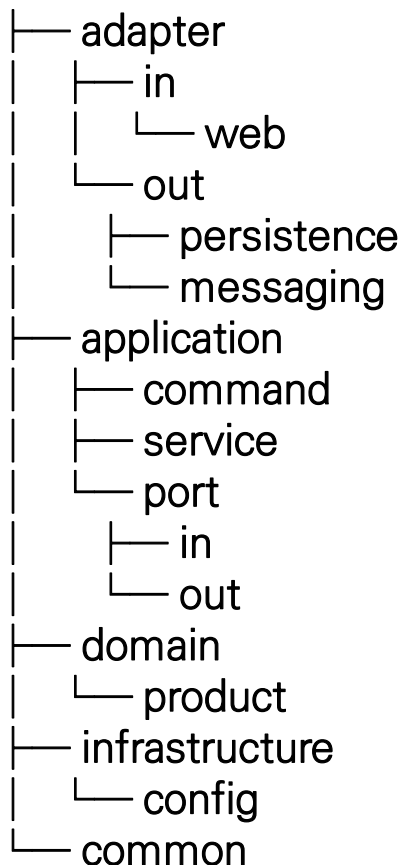
계층형 사례

```
product-service
├── src/main/java/com/cretec/product
│   ├── controller
│   ├── service
│   ├── domain
│   ├── repository
│   ├── dto
│   ├── event
│   ├── config
│   └── common
```

```
controller
├── ProductController.java
service
├── ProductService.java
domain
├── Product.java
├── ProductImage.java
├── ProductAttributeValue.java
└── ProductStatus.java
repository
├── ProductRepository.java
dto
├── ProductCreateRequest.java
├── ProductResponse.java
└── ProductUpdateRequest.java
event
├── ProductRegisteredEvent.java
└── ProductEventPublisher.java
```

Hexagonal 사례

product-service



adapter/in/web



application/port/in



application/port/out



application/command



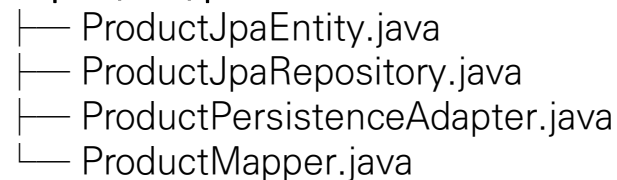
application/service



domain/product



adapter/out/persistence



adapter/out/messaging



클래스 역할 정의

- 클래스 역할, 금지, 책임수준 표준화

클래스/구성요소	책임	금지
Controller	HTTP 요청 수신, Request DTO 검증, Command 변환, Response 반환	비즈니스 로직, 트랜잭션 처리, Repository 직접 호출
Request DTO	외부 요청 데이터 표현, 기본 형식 검증	비즈니스 규칙 처리, Domain 객체 역할
Response DTO	외부 응답 데이터 표현	Domain/JPA Entity 직접 노출
Command	유스케이스 실행에 필요한 입력값 전달	HTTP/JPA/Kafka 의존
Application Service	유스케이스 실행, 트랜잭션 관리, Aggregate 조회/저장, 이벤트 발행 조율	복잡한 도메인 규칙 직접 구현, ⇒ 도메인 서비스에서 HTTP 처리, JPA 상세 구현

클래스 역할 정의

클래스/구성요소	책임	금지
Aggregate Root	핵심 비즈니스 규칙, 상태 변경, 내부 Entity 일관성 관리	DB 접근, Kafka 발행, 외부 API 호출, Spring 의존
Entity	식별자를 가진 도메인 객체, Root 통제 하에 상태 보유	Root 우회 수정, 외부 직접 접근
Value Object	값 표현, 값 자체의 유효성 검증, 불변성 유지	식별자 보유, DB/API 직접 처리
Domain Service	Aggregate 내부만으로 처리하기 어려운 도메인 규칙 처리	트랜잭션 흐름 제어, Controller 역할, 인프라 처리
Repository Interface	Aggregate 단위 조회/저장 추상화	비즈니스 규칙 처리, 내부 Entity별 무분별한 저장
Persistence Adapter / Repository Impl	JPA/MyBatis 등 실제 DB 접근 구현	도메인 규칙 판단, 유스케이스 흐름 제어
JPA Entity	DB 테이블 매핑	API 응답 직접 노출, 복잡한 비즈니스 로직 집중

클래스 역할 정의

클래스/구성요소	책임	금지
Mapper	DTO↔Command, Domain↔JPA Entity 변환	비즈니스 판단, 외부 호출
Domain Event	도메인 내부 사건 표현	Kafka 전송 포맷과 강결합
Integration Event	외부 서비스 전달용 이벤트 계약	내부 도메인 모델 그대로 노출
Event Publisher	이벤트 발행, Kafka/Outbox 연계	도메인 상태 변경, 유스케이스 조율
Event Consumer	외부 이벤트 수신, Command 변환 후 Application 호출	복잡한 비즈니스 로직 직접 처리
Exception Handler	공통 예외 응답 변환	비즈니스 규칙 판단
Config	Bean, Security, Kafka, DB 등 기술 설정	도메인 규칙 포함

DTO(Data Transfer Object)/Command 규칙

- ✓ DTO는 데이터 전달용 객체: ex) RequestDTO, ResponseDTO
- ✓ DTO는 API 계약에 해당. 프론트/외부 시스템과 약속된 데이터 형식
- ✓ Command: 하나의 유스케이스 실행 요청. 단순 데이터 아닌 유스케이스 실행 명령
- ✓ 외부 요청 데이터와 내부 유스케이스 데이터를 어떻게 분리할지를 결정

항목	DTO	Command
목적	데이터 전달	유스케이스 실행
관심사	API/외부 통신	Application 내부
위치	Controller/Adapter	Application
의미	데이터 구조	행동 요청
예	ProductRequest	RegisterProductCommand

DTO(Data Transfer Object)/Command 규칙

- ✓ DTO와 Command 분리 이유는
 - 외부 API 변경으로부터 내부 유스케이스 보호
 - 유스케이스 명확화
 - 도메인 보호



DTO(Data Transfer Object)/Command 규칙

외부요청(JSON)

```
{  
  "productCode": "P-1001",  
  "productName": "전동드릴",  
  "basePrice": 10000  
}
```

Request DTO

```
public record RegisterProductRequest(  
    String productCode,  
    String productName,  
    BigDecimal basePrice  
) { }
```

Command 변환

```
public RegisterProductCommand toCommand() {  
    return new RegisterProductCommand(  
        productCode,  
        productName,  
        basePrice  
    );  
}
```

Application Service

```
productApplicationService.register(  
    command);
```

Domain

```
Product.register(...)
```

DTO(Data Transfer Object)/Command 규칙

- ✓ 단순 CRUD, API 입력값과 유스케이스 입력값이 거의 같거나, 내부 시스템/교육용/초기 개발이면 DTO/Command 하나로 가도 됨
- ✓ 하지만 시스템이 복잡해지거나 아래 상황이면 분리해야 함

상황 변화	문제
API 필드명이 바뀔	Application Service 영향
모바일/웹 요청 형식이 다름	서비스가 외부 형식에 끌려감
Kafka 이벤트로도 같은 유스케이스 호출	REST DTO 재사용이 어색함
Command에 도메인 타입을 쓰고 싶음	DTO와 충돌
API 검증과 업무 검증을 나누고 싶음	역할 혼재

레포지토리 규칙

- ✓ Repository: Aggregate를 저장/조회하는 컬렉션 같은 추상화
- ✓ Repository는 Aggregate Root 기준으로 만들어야 함
 - Aggregate Root가 일관성을 관리하기 위해
- ✓ Repository는 비즈니스 로직을 가지지 않음
- ✓ Repository는 Aggregate를 완전한 형태로 반환
- ✓ 저장 단위는 Aggregate 전체
- ✓ Repository 인터페이스 위치
 - domain/repository
 - application/port/out

레포지토리 규칙

- ✓ 다른 Aggregate 조인 남용 금지
 - 서비스 경계 침범, Aggregate 경계 붕괴
- ✓ Repository는 “도메인 계층에 정의된 Aggregate 저장 인터페이스”
- ✓ Infrastructure 기술 숨김:
 - JPA, MyBatis, Mongo, Redis etc.
- ✓ Transaction은 Repository가 아니라 Application Service
- ✓ Query Repository 분리 가능(CQRS)

이벤트 발행 방식

- ✓ 이벤트 종류 규칙
 - Domain Event
 - Integration Event
- ✓ 이벤트 Payload 구조 표준

필드	의미
eventId	이벤트 고유 ID
eventType	이벤트명
eventVersion	버전
occurredAt	발생시간
source	발생 서비스
correlationId	Saga 추적
payload	실제 데이터

이벤트 발행 방식

✓ 이벤트 Payload 구조 사례

```
{
  "eventId": "uuid",
  "eventType": "ProductRegistered",
  "eventVersion": "v1",
  "occurredAt": "2026-05-18T10:00:00Z",
  "source": "product-service",
  "correlationId": "trace-id",
  "payload": {
    "productId": 1001,
    "productCode": "P-1001"
  }
}
```

이벤트 발행 방식

- ✓ Topic Naming 규칙
 - {domain}.{event-name}.{version}
 - product.product-registered.v1
 - sales.order-created.v1
 - inventory.stock-reserved.v1

이벤트 발행 위치

- ✓ 도메인이 기술적 내용 알지 못하도록 해야 함

좋은 사례

Aggregate
→ Domain Event 생성
→ Application Service
→ Repository Save
→ Outbox 저장
→ Kafka 발행

이벤트 발행 시점

- ✓ DB commit 이후 발행
 - 👉 Rollback 이벤트 방지

Consumer 표준

- ✓ 다음 사항들 정해야 함
 - Retry
 - DLQ
 - 멍등성
 - Error Handling

예외 처리

항목	결정 내용
예외 유형	BusinessException, NotFoundException, DuplicateException, ValidationException, ExternalSystemException
예외 발생 위치	Domain/Application/Infrastructure 중 어디서 발생할지
에러 코드 체계	PRODUCT_NOT_FOUND, PRODUCT_DUPLICATED, ORDER_INVALID_STATUS
HTTP Status 매핑	400, 401, 403, 404, 409, 500 등
공통 응답 포맷	성공/실패 응답 JSON 구조
메시지 규칙	사용자 메시지와 내부 로그 메시지 구분
Validation 처리	DTO 검증 오류를 어떻게 반환할지
외부 시스템 오류	Kafka/API/DB 장애를 어떤 예외로 감쌀지
로깅 수준	WARN/ERROR 기준
추적 ID	traceld, correlationId 포함 여부

예외 처리

공통오류 응답 사례

```
{  
  "success": false,  
  "code": "PRODUCT_DUPLICATED",  
  "message": "이미 존재하는  
상품코드입니다.",  
  "traceId": "abc-123"  
}
```

예외 클래스 사례

```
public class BusinessException extends  
    RuntimeException {  
    private final String errorCode;  
  
    public BusinessException(String errorCode,  
        String message) {  
        super(message);  
        this.errorCode = errorCode;  
    }  
  
    public String getErrorCode() {  
        return errorCode;  
    }  
}
```


예외 처리

위치별 원칙

- DTO → 형식 검증 오류
- Domain → 비즈니스 규칙 위반
- Application → 유스케이스 수행 불가
- Infrastructure → DB/API/Kafka 장애
- ControllerAdvice → 최종 HTTP 응답 변환

금지사항

- Controller에서 try-catch 남발 금지
- Repository에서 비즈니스 예외 판단 금지
- Domain에서 HTTP Status 사용 금지
- 예외 메시지에 DB/서버 내부정보 노출 금지
- 서비스마다 에러 응답 포맷 다르게 만들기 금지

테스트 방식: 종류

테스트	대상	목적
Domain Unit Test	Aggregate, VO, Domain Service	비즈니스 규칙 검증
Application Service Test	유스케이스	흐름, 트랜잭션, 이벤트 발생 검증
Repository Integration Test	DB/JPA/MyBatis	저장/조회, 매핑 검증
API Test	Controller	요청/응답, Validation, Error 응답 검증
Event Test	Kafka Producer/Consumer	이벤트 발행/수신, Payload 검증
Contract Test	서비스 간 API/Event 계약	타 서비스 연동 깨짐 방지
E2E Test	전체 업무 흐름	주문→재고→배송 등 통합 검증
Performance Test	API/Event/DB	처리량, 응답시간 검증

테스트 방식: 우선 순위 결정

1. Domain 규칙 테스트 -> 가장 우선적이고 중요함
2. Application 유스케이스 테스트
3. Repository/API 통합 테스트
4. Event/Contract 테스트
5. E2E/성능 테스트

```
@Test
void 상품명_누락시_400을_반환한다() throws Exception {
    mockMvc.perform(post("/products")
        .contentType(MediaType.APPLICATION_JSON)
        .content("{}"))
        .andExpect(status().isBadRequest());
}
```

테스트 방식: 표준으로 설정할 것

- ✓ 테스트 명명 규칙
- ✓ Mock 사용 기준
- ✓ Testcontainers 사용 여부: 실제 DB/Kafka를 컨테이너로 띄워 통합 테스트할지 결정
- ✓ Kafka 테스트 방식
- ✓ DB 테스트 데이터 초기화 방식: 테스트마다 동일한 DB 상태를 보장하는 방법
- ✓ CI에서 필수 통과 기준
- ✓ 커버리지 기준: 전체 70~80% 이상, 핵심 도메인은 80~90% 이상 권장
- ✓ Contract Test 적용 대상: 서비스 간 API/이벤트 계약 검증 대상

개념

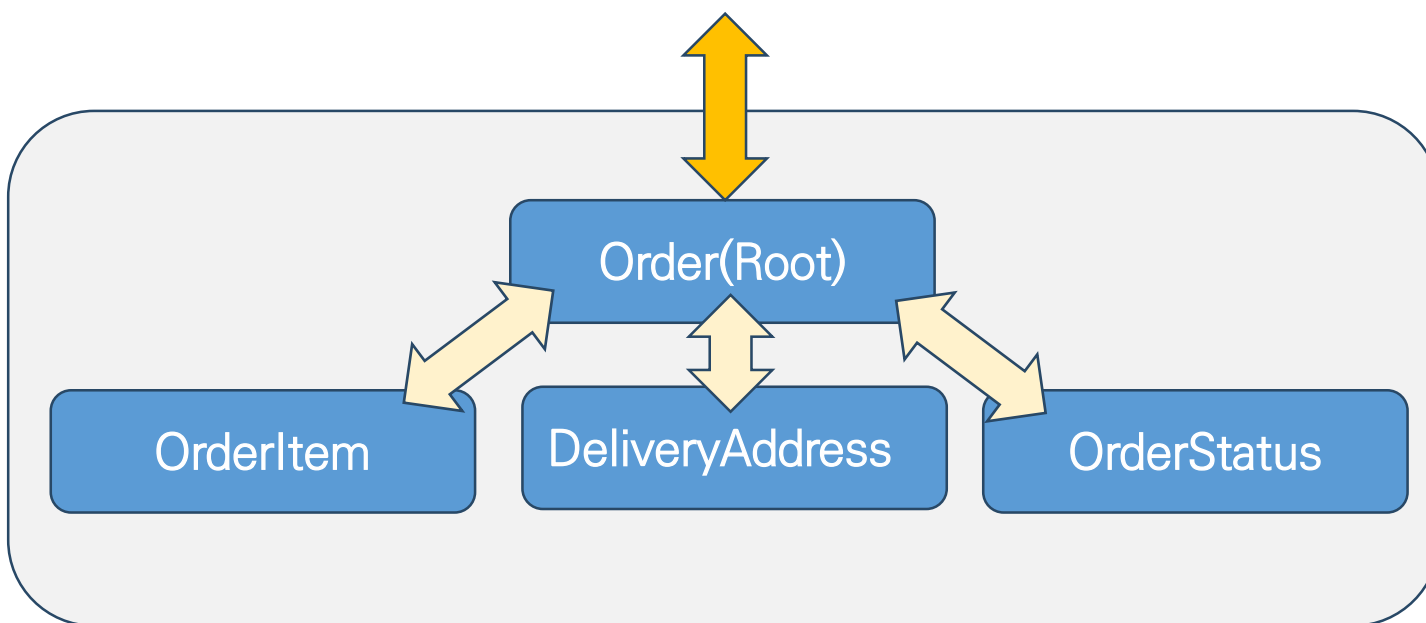
- ✓ Aggregate 구현 설계는 DDD에서 도출한 Aggregate를 실제 코드, DB, 트랜잭션 구조로 바꾸는 설계
 - Aggregate Root 중심으로 상태 변경, 업무 규칙, 트랜잭션, 저장 단위를 확정
- ✓ Aggregate는 “업무 규칙을 지키면서 한 번에 저장·변경되어야 하는 객체 묶음”이고, Aggregate Root는 그 묶음의 대표 출입구

Aggregate 구현 설계 과정

단계	설계 내용	산출물
1	Aggregate 후보 식별	주문, 결제, 배송, 재고 등 후보 목록
2	Root 결정	Order, Payment, Inventory 같은 대표 엔티티
3	내부 Entity/Value Object 구분	OrderItem, Money, Address
4	업무 규칙 정의	취소 가능 조건, 수량 제한, 상태 전이 규칙
5	상태 변경 메서드 설계	cancel(), pay(), ship()
6	트랜잭션 경계 확정	한 트랜잭션에서 변경할 객체 범위
7	다른 Aggregate 참조 방식 결정	객체 참조 금지, customerId, productId로 참조
8	Repository 설계	Root 단위 저장/조회
9	도메인 이벤트 설계	OrderCreated, PaymentCompleted
10	JPA/DB 매핑 설계	테이블, FK, Embeddable, Cascade 범위

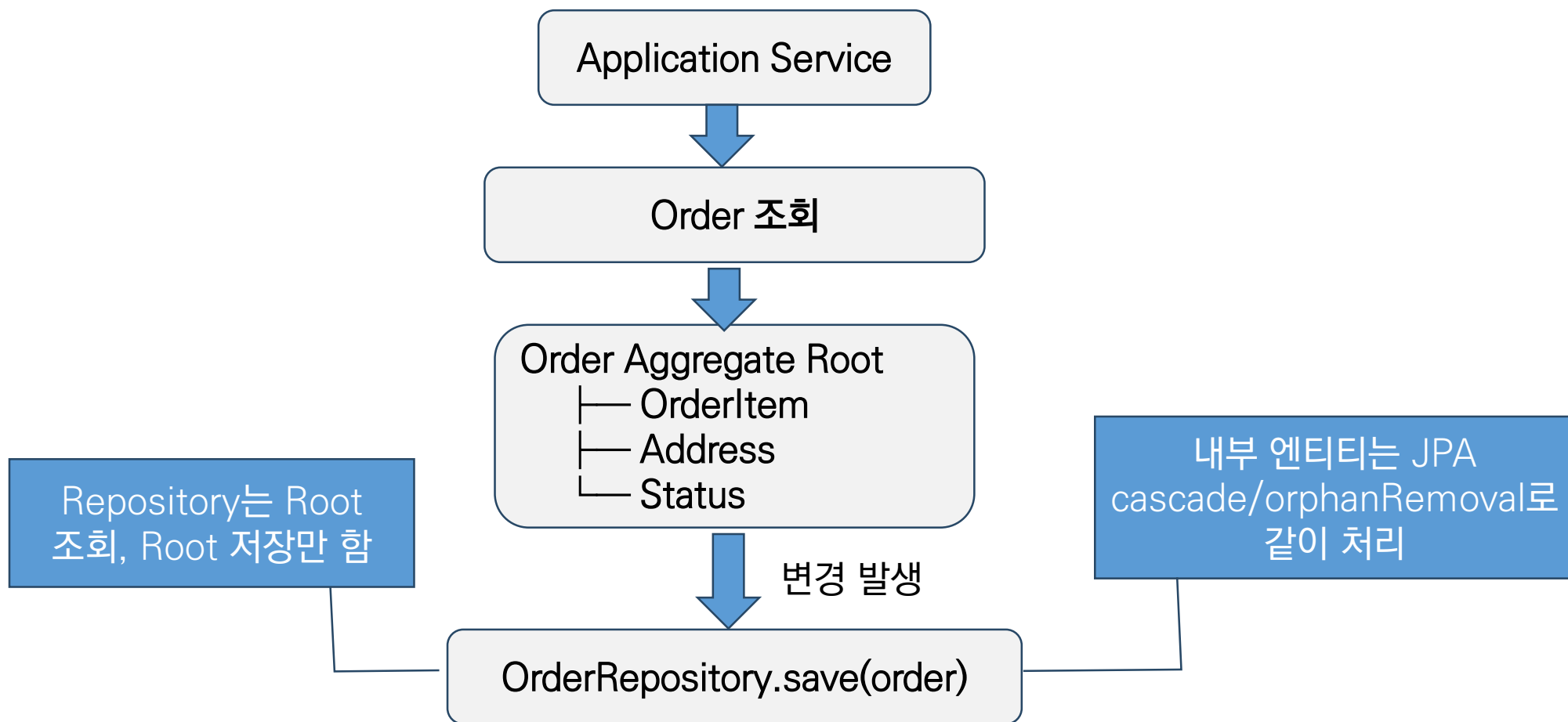
Aggregate root 통한 엔티티 수정 사례

- ✓ 외부는 내부 엔티티 직접 수정 금지
- ✓ 반드시 Order Root를 통해 수정
- ✓ Root가 업무 규칙 검증 수행



Order Aggregate

Aggregate root 통한 엔티티 수정 사례



Aggregate root 통한 엔티티 수정 사례: Root

```
@Entity
public class Order {
    @Id
    private Long id;
    @OneToMany(mappedBy = "order",
        cascade = CascadeType.ALL,
        orphanRemoval = true
    )
    private List<OrderItem> items = new ArrayList<>();

    @Enumerated(EnumType.STRING)
    private OrderStatus status;

    public void changeItemQuantity( Long productId, int quantity) {
        OrderItem item = items.stream()
            .filter(i -> i.getProductId().equals(productId))
            .findFirst().orElseThrow();
        item.changeQuantity(quantity);
    }
}
```

Order에 의해 매핑됨. FK
주입

Order 저장/삭제 시
내부 OrderItem도 같이 처리

컬렉션에서 제거된 자식
엔티티 자동 삭제

Aggregate root 통한 엔티티 수정 사례: 내부 엔티티

FK 가진 쪽이 연관관계 주인

OrderItem 조회 시
Order를 즉시 가져오지 말라

```
@Entity
public class OrderItem {
```

```
    @Id
    @GeneratedValue
    private Long id;
```

```
    private Long productId;
```

```
    private int quantity;
```

```
    @ManyToOne(fetch = FetchType.LAZY)
    private Order order;
```

```
    public void changeQuantity(int quantity) {
        this.quantity = quantity;
    }
```

```
    public Long getProductId() {
        return productId;
    }
```

```
}
```

OrderItem은 Order Aggregate Root에 소속되며, DB에서는 order_id
FK로 연결되고, 실제 Order 객체는 필요할 때만 조회

Aggregate root 통한 엔티티 수정 사례: 서비스

```
@Service
@Transactional
public class OrderService {

    private final OrderRepository
    orderRepository;
```

```
public void changeQuantity(Long orderId, Long
productId, int quantity ) {
```

```
    Order order = orderRepository.findById(orderId)
    .orElseThrow();
```

```
    // Root 통해 내부 엔티티 수정
    order.changeItemQuantity(productId, quantity );
```

```
    // save 안해도 dirty checking 가능
}
```

쓰기는 Root만, 읽기는 자유롭게

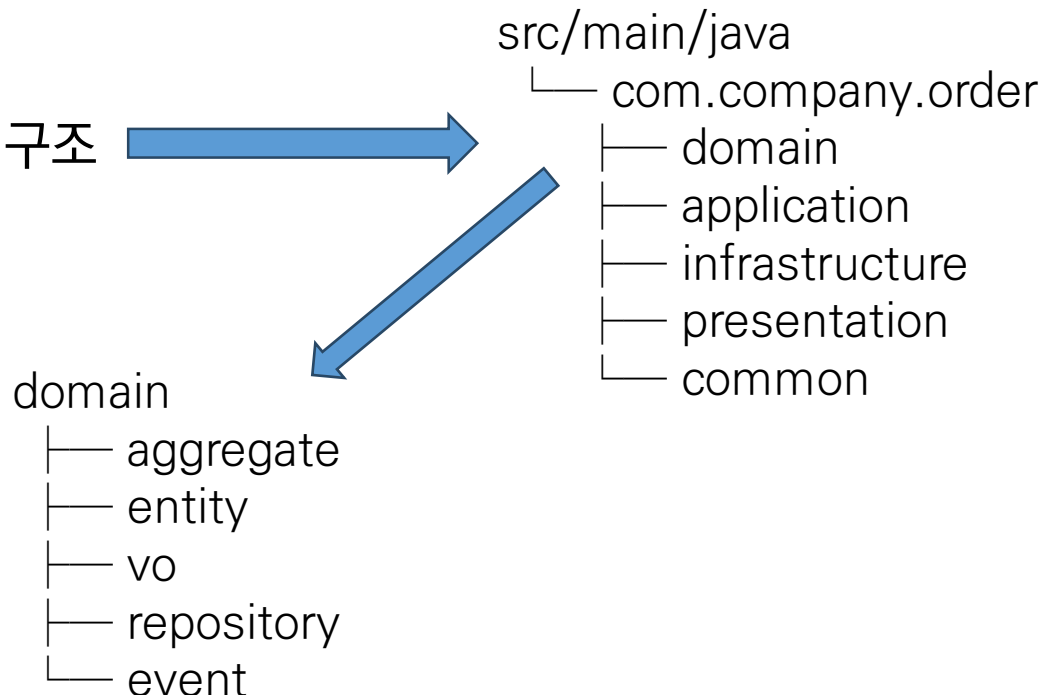
엔티티 객체의 값이
변경되면 JPA가 이를
자동 감지해서 UPDATE
SQL을 실행하는 기능

개념

- ✓ 앞 단계에서 정의한 아키텍처/DDD/패키지 규칙을 기반으로 실제 개발 가능한 “서비스 기본 골격(boilerplate)”을 만드는 단계
- ✓ 모든 서비스 구조 동일, 신규 서비스 즉시 생성, 예외/로그/보안 자동 적용

구성 요소

- ✓ 기본 프로젝트 구조



구성 요소

- ✓ 공통 기술 설정
- ✓ 공통 예외 처리
- ✓ 공통 응답 포맷
- ✓ 공통 로깅
- ✓ 기본 CI/CD
- ✓ 기본 테스트 구조

핵심 작업

해야 할 일	의미
Aggregate Root 클래스 작성	예: Order, Payment, Inventory
내부 Entity 작성	예: OrderItem, PaymentHistory
Value Object 작성	예: Money, Address, Quantity
상태값 Enum 작성	예: OrderStatus, PaymentStatus
생성 메서드 작성	잘못된 상태로 객체 생성되지 않게 함 - 유효성 검증, 초기 상태 보장
상태 변경 메서드 작성	cancel(), changeQuantity(), completePayment()
업무 규칙 검증	수량 음수 금지, 취소 후 수정 금지 등
외부 setter 제거	아무나 상태를 바꾸지 못하게 함
내부 엔티티 접근 제한	Root를 통해서만 수정
Repository 인터페이스 작성	Root 단위 저장/조회
도메인 이벤트 발행	OrderCreatedEvent, OrderCancelledEvent

개념

- ✓ DDD 객체 모델 ↔ RDB 테이블/JPA 매핑 연결

작업	설명
Entity 매핑	@Entity, @Table
PK 설계	@Id, @GeneratedValue
연관관계 매핑	@OneToMany, @ManyToOne
FK 설계	@JoinColumn
Value Object 매핑	@Embeddable, @Embedded
Enum 저장 방식	@Enumerated
Cascade 전략	Aggregate 내부 저장 전파

개념

- ✓ DDD 객체 모델 ↔ RDB 테이블/JPA 매핑 연결

작업	설명
orphanRemoval 설정	내부 엔티티 자동 삭제
Fetch 전략	LAZY: 즉시 패치 안함 EAGER: 연관 객체를 즉시 같이 조회
Repository 구현	JpaRepository
트랜잭션 전략	@Transactional 👉 메소드를 하나의 트랜잭션 처리
Dirty Checking 활용	객체 변경 자동 UPDATE
테이블 구조 설계	aggregate별 테이블 구성
인덱스/성능 설계	조회 최적화
낙관적 락	@Version

개념

✓ Embeddable/Embedded

어노테이션	붙는 위치	역할
@Embeddable	값 객체 클래스	“이 클래스는 내장 가능 VO이다” 선언
@Embedded	사용하는 필드	“이 VO를 현재 Entity에 포함한다” 선언

이 클래스는 별도 테이블이 아니라
다른 Entity 안에 포함되어 저장된다

```
@Embeddable
public class Address {

    private String city;
    private String street;
    private String zipcode;
}
```

Entity 안에서 실제 사용하는 곳에
붙임

```
@Entity
public class Order {

    @Id
    private Long id;

    @Embedded
    private Address address;
}
```

개념

✓ Embedded/Entity

구분	Embedded VO	Entity
ID	없음	있음
독립 저장	불가	가능
생명주기	부모 소속	독립
테이블	부모와 합쳐짐	별도
변경	교체 중심	상태 변경 중심

개념

✓ @JoinColumn: FK 지정

```
@Entity
public class Order {

    @Id
    @GeneratedValue
    private Long id;

    @OneToMany(mappedBy = "order",
                cascade = CascadeType.ALL)
    private List<OrderItem> items = new
    ArrayList<>();
}
```

```
@Entity
public class OrderItem {

    @Id
    @GeneratedValue
    private Long id;

    private Long productId;

    private int quantity;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "order_id")
    private Order order;
}
```

개념

- ✓ @Version: 낙관적 Lock 사용

동시 commit을 낙관적으로 허용하되 @Version 정보를 JPA가 검사하여 최종 commit 여부 결정

```
@Entity
public class Product {

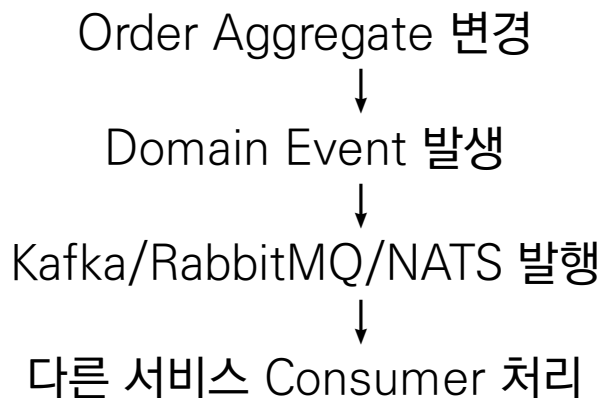
    @Id
    private Long id;

    private int stock;

    @Version
    private Long version;
}
```

핵심

- ✓ Aggregate 내부 변경을 외부 서비스에 안전하게 전파하는 구조를 만드는 것
 - 예: Order 상태 변경 → Payment/Inventory/Delivery 서비스에 전달



핵심 구현 요소

요소	역할
Domain Event	Aggregate 내부 상태 변경 표현
Event Publisher	이벤트 발행
Message Broker	Kafka/RabbitMQ/NATS
Producer	메시지 전송
Consumer	메시지 수신 처리
Event Payload	JSON 메시지 구조
Topic/Queue 설계	이벤트 채널 정의
Outbox Pattern	DB와 메시지 일관성 보장
CDC	Outbox 이벤트 추출
Idempotency	중복 처리 방지
Retry/DLQ	실패 복구
Event Versioning	메시지 스키마 버전 관리
Saga 처리	서비스 간 분산 트랜잭션
Observability	traceld/log 추적

Event Publisher

```
@Component
public class OrderEventPublisher {

    private final KafkaTemplate<String, Object> kafkaTemplate;

    public void publish(OrderCreatedEvent event) {

        kafkaTemplate.send(
            "order.created",
            event
        );
    }
}
```

Kafka Topic

```
order.created
order.cancelled
payment.completed
inventory.decreased
```

Event Consumer

```
@KafkaListener(topics = "order.created")
public void consume(OrderCreatedEvent event) {

    inventoryService.decreaseStock(
        event.getOrderId()
    );
}
```


Outbox Table

outbox_event

id

aggregate_type

aggregate_id

event_type

payload

created_at

published

Idempotency

- ✓ 같은 이벤트 여러 번 와도 한 번만 처리 가능해야 함

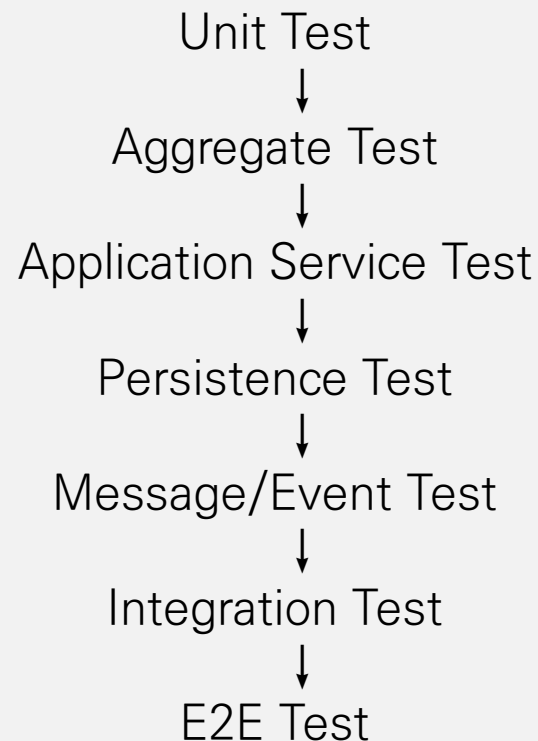
```
if (alreadyProcessed(eventId)) {  
    return;  
}
```

중요한 설계 포인트

중요 요소	이유
Outbox	데이터 정합성
Idempotency	중복 처리
DLQ	장애 복구
Event schema	서비스 계약
Saga	분산 트랜잭션
TraceId	운영 추적: 보통 OpenTelemetry로 trace 정보 수합 후 Jaeger나 Tempo에서 확인

개념

- ✓ Aggregate, 서비스, 이벤트, 메시징이 설계한 규칙대로 실제 안전하게 동작하는지 검증하는 것
- ✓ DDD/MSA에서는 단순 기능 테스트보다 도메인 규칙, 트랜잭션, 이벤트 흐름, 서비스 간 정합성 검증이 핵심



핵심 테스트 요소

요소	검증 대상
Aggregate Test	도메인 규칙
Entity/VO Test	값 검증
Service Test	유스케이스 흐름
Repository Test	JPA 매핑
Transaction Test	rollback/commit
Event Test	이벤트 발생
Kafka/RabbitMQ Test	메시징 흐름

핵심 테스트 요소

요소	검증 대상
Saga Test	분산 트랜잭션
Idempotency Test	중복 메시지
Retry/DLQ Test	장애 처리
API Test	REST 계약
Integration Test	서비스 연동
Performance Test	TPS/응답시간
Contract Test	이벤트 스키마 호환성

Aggregate 테스트

```
@Test
void 배송완료된_주문은_취소할수없다() {

    Order order = Order.create(...);

    order.ship();

    assertThrows(
        IllegalStateException.class,
        order::cancel
    );
}
```



업무규칙 테스트

VO 테스트

```
@Test
void 금액은_음수가될수없다() {

    assertThrows(
        IllegalArgumentException.class,
        () -> new Money(-1000)
    );
}
```



VO 규격, 룰 테스트

테스트 원칙

원칙	설명
도메인 우선	Aggregate 규칙 집중 검증
DB 없는 테스트 우선	빠른 단위 테스트
외부 의존 최소화	mock/stub 활용
이벤트 검증 필수	MSA 핵심
실패 시나리오 중요	rollback/retry
읽기보다 쓰기 중요	Aggregate 보호

도메인 규칙이 절대 깨지지
않음을 증명하는 작업

개념

- ✓ 개발(Dev)부터 운영(Ops)까지 배포를 자동화하고 안정적으로 반복 가능하게 만드는 것
 - ✓ 코드 작성 → 빌드 → 테스트 → 컨테이너화 → 배포 → 운영 → 모니터링 자동화
-
- | | |
|--|---|
| <ul style="list-style-type: none">✓ 자동화<ul style="list-style-type: none">• 수동 배포 최소화✓ 재현 가능성<ul style="list-style-type: none">• 언제든지 동일 환경 생성 가능✓ 롤백 가능성<ul style="list-style-type: none">• 실패 시 즉시 복구 | <ul style="list-style-type: none">✓ 관측 가능성<ul style="list-style-type: none">• 문제 원인 추적 가능✓ 선언형 운영<ul style="list-style-type: none">• YAML/Git 기반 |
|--|---|

Q & A